Lesson 3 Teacher's Key

TIC-80

INSTRUCTOR ONLY - do not distribute to students

Level 1 - Complete Worked Solution

A bullets table, a spawn check, and an update/cleanup loop, all inside the existing TIC() function:

```
bullets={}

function TIC()
 cls(0)
 if btn(3) then x=x+1 end
 if btnp(4) then
  table.insert(bullets,{bx=x,by=y})
 end
 for i=#bullets,1,-1 do
  local b=bullets[i]
  b.by=b.by-2
  if b.by<0 then
   table.remove(bullets,i)
  else
   spr(2,b.bx,b.by,0)
  end
 end
 spr(1,x,y,0)
end
```

Level 2 - Complete Worked Solution

btnp() is already edge-triggered (true for exactly one frame per physical press, confirmed against TIC-80's own documentation - it's specifically designed as the "just pressed" check, as opposed to btn()'s "currently held"), so Level 1's fire check needs no changes for Level 2.

Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking.

Sound: play on fire

Design a short descending-pitch effect in the SFX Editor (F4) in an empty slot (e.g. slot 1), then call sfx(1) right next to the table.insert() call - a single call, not looped, so it's fire-and-forget the same way Lesson 2's beep-beep idea needed an explicit sfx(-1) stop but a one-shot fire sound doesn't:

```
if btnp(4) then
 table.insert(bullets,{bx=x,by=y})
 sfx(1)
end
```

Visual: give the bullet its own look

spr()'s flip argument (the 5th parameter) can give the bullet a rotating look as it travels, using the frame counter t() for a changing value each frame - drawn on a distinct sprite index (2) from the player (1) already satisfies most of "its own look" on its own:

```
for i=#bullets,1,-1 do
 local b=bullets[i]
 b.by=b.by-2
 if b.by<0 then
  table.remove(bullets,i)
 else
  spr(2,b.bx,b.by,0,1,math.floor(t()/4)%4)
 end
end
```

Structure: keep the two loops separate

The spawn check (btnp(4) + table.insert) and the update/cleanup loop (movement, drawing, sound, visual) are two distinct pieces, shown as two separate blocks above - resist the temptation to fold bullet-spawning logic into the same lines that check player movement, or vice versa, even though they all live inside the same TIC() function by necessity (TIC-80 only has the one entry-point function).

Complete Level 3 solution (all 3 goals combined)

Everything above combined into the one TIC() function:

```
bullets={}

function TIC()
 cls(0)
 if btn(3) then x=x+1 end
 if btnp(4) then
  table.insert(bullets,{bx=x,by=y})
  sfx(1)
 end
 for i=#bullets,1,-1 do
  local b=bullets[i]
  b.by=b.by-2
  if b.by<0 then
   table.remove(bullets,i)
  else
   spr(2,b.bx,b.by,0,1,math.floor(t()/4)%4)
  end
 end
 spr(1,x,y,0)
end
```